Parallel Mater is a physics API offering functions and structures that allow you to define physics simulations. Parallel Mater supports Rigid Bodies, Fluid, Cloth, Rope, Smoke, and Soft Bodies. Parallel Mater uses a configured fixed-function pipeline to simulate the specified physics simulations and their specified couplings.

Parallel Mater

Function names: pmCreateInstance
Structure and type names: PmInstanceCreateInfo
Constants and return codes: PM_SUCCESS
PFN_pmName: pointer type for pmName
[n.n] Specification section

Return codes [2]

Success & error codes [2.1]

Return codes are reported via PmResult return values.

```
typedef enum PmResult {
    PM_SUCCESS = 0,
    PM_NOT_READY = 1,
    PM_TIMEOUT = 2,
    PM_INCOMPLETE = 3,
    PM_ERROR_INVALID_ARGUMENT = -1,
    PM_ERROR_INVALID_STATE = -2,
    PM_ERROR_OUT_OF_HOST_MEMORY = -3,
    PM_ERROR_OUT_OF_DEVICE_MEMORY = -4,
    PM_ERROR_FEATURE_NOT_PRESENT = -5,
    PM_ERROR_FORMAT_NOT_SUPPORTED = -6,
    PM_ERROR_CAPACITY_EXCEEDED = -7,
    PM_ERROR_DEVICE_LOST = -8,
    PM_ERROR_INCOMPATIBLE_VERSION = -9,
    PM_ERROR_DRIVER_NOT_FOUND = -10,
    PM_ERROR_LAYER_NOT_PRESENT = -11,
    PM_ERROR_EXTENSION_NOT_PRESENT = -12
} PmResult;
```

Function Pointers and Instancing [3]

Function pointers [3.1]

```
typedef void (PMAPI_CALL
  *PFN_pmVoidFunction)(
    void);
PFN_pmVoidFunction pmGetInstanceProcAddr(
    PmInstance instance, const char* pName);
PFN_pmVoidFunction pmGetDeviceProcAddr(
    PmDevice device, const char* pName);
```

Instance creation [3.2]

```
typedef struct PmLayerProperties {
    char layerName[256];
    uint32_t specificationVersion;
    uint32_t implementationVersion;
    char description[256];
} PmLayerProperties;
typedef struct PmExtensionProperties {
    char extensionName[256];
    uint32_t specificationVersion;
} PmExtensionProperties;
typedef struct PmInstanceCreateInfo {
    PmStructureType sType;
    const void* pNext;
    uint32_t apiVersion;
    const char* pApplicationName;
    PmBool32 enableValidation;
    uint32_t enabledLayerCount;
    const char* const* ppEnabledLayerNames;
    uint32_t enabledExtensionCount;
    const char* const*
      ppEnabledExtensionNames;
} PmInstanceCreateInfo;
PmResult pmEnumerateInstanceVersion(
    uint32_t* pApiVersion);
PmResult pmEnumerateInstanceLayerProperties(
    uint32_t* pCount, PmLayerProperties*
      pProperties);
PmResult
  pmEnumerateInstanceExtensionProperties(
    const char* pLayerName, uint32_t*
      pCount,
    PmExtensionProperties* pProperties);
PmResult
  pmEnumerateDeviceExtensionProperties(
    PmPhysicalDevice physicalDevice, const
      char* pLayerName,
    uint32_t* pCount, PmExtensionProperties*
      pProperties);
PmResult pmCreateInstance(
    const PmInstanceCreateInfo* pInfo,
    const PmAllocationCallbacks* pAllocator,
    PmInstance* pInstance);
void pmDestroyInstance(PmInstance instance);
```

Structured validation diagnostics [3.3]

```
#define
  PM_VALIDATION_RULE_INVALID_LAYER_CALL
  "PM-VAL-0001"
#define PM_VALIDATION_RULE_INVALID_ARGUMENT
  "PM-VAL-0002"
```

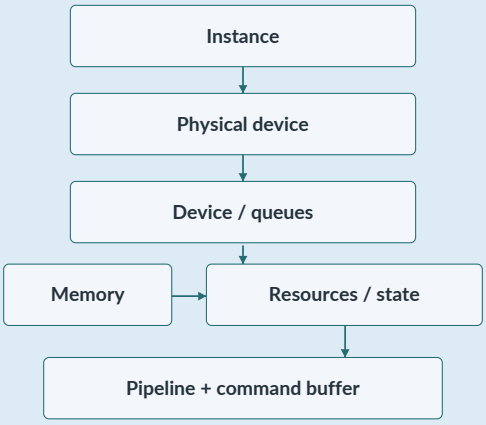
Structured validation diagnostics (continued) [3.3]

```
#define PM_VALIDATION_RULE_INVALID_STATE
  "PM-VAL-0003"
#define
  PM_VALIDATION_RULE_TRACKING_UNAVAILABLE
  "PM-VAL-0004"
typedef enum PmValidationSeverity {
    PM_VALIDATION_SEVERITY_WARNING = 1,
    PM_VALIDATION_SEVERITY_ERROR = 2
} PmValidationSeverity;
typedef enum PmValidationObjectType {
    PM_VALIDATION_OBJECT_UNKNOWN = 0,
    PM_VALIDATION_OBJECT_INSTANCE = 1,
    PM_VALIDATION_OBJECT_PHYSICAL_DEVICE =
      2,
    PM_VALIDATION_OBJECT_DEVICE = 3,
    PM_VALIDATION_OBJECT_QUEUE = 4,
    PM_VALIDATION_OBJECT_COMMAND_BUFFER = 5
} PmValidationObjectType;
typedef struct PmValidationDiagnostic {
    const char* pRuleId;
    PmValidationSeverity severity;
    PmResult result;
    const char* pCommandName;
    PmValidationObjectType objectType;
    const void* object;
    uint32_t argumentSize;
    const void* pArguments;
    const char* pMessage;
} PmValidationDiagnostic;
typedef void (PMAPI_CALL
  *PmValidationDiagnosticFn)(
    void* pUserData,
    const PmValidationDiagnostic*
      pDiagnostic);
typedef struct
  PmValidationDiagnosticsCreateInfo {
    PmStructureType sType;
    const void* pNext;
    PmValidationDiagnosticFn pfnDiagnostic;
    void* pUserData;
} PmValidationDiagnosticsCreateInfo;
```

Host allocation callbacks [3.4]

```
typedef void* (PMAPI_CALL *PmAllocateFn)(
    void* user, size_t size, size_t
      alignment);
typedef void (PMAPI_CALL *PmFreeFn)(
    void* user, void* allocation);
typedef struct PmAllocationCallbacks {
    void* pUserData;
    PmAllocateFn pfnAllocate;
    PmFreeFn pfnFree;
} PmAllocationCallbacks;
```

Devices & queues [4]



allocate → bind → record → submit

Physical devices [4.1]

```
PmResult pmEnumeratePhysicalDevices(
    PmInstance instance, uint32_t* pCount,
    PmPhysicalDevice* pDevices);
```

Physical-device identity [4.2]

```
typedef struct PmPhysicalDeviceProperties {
    uint32_t apiVersion;
    uint32_t driverVersion;
    uint32_t vendorId;
    uint32_t deviceId;
    uint8_t deviceUuid[16];
    char deviceName[256];
} PmPhysicalDeviceProperties;
void pmGetPhysicalDeviceProperties(
    PmPhysicalDevice physicalDevice,
    PmPhysicalDeviceProperties*
      pProperties);
```

Queue families [4.3]

```
enum {
    PM_QUEUE_PHYSICS_BIT = 0x1,
    PM_QUEUE_TRANSFER_BIT = 0x2
};
typedef struct PmQueueFamilyProperties {
    PmFlags queueFlags;
    uint32_t queueCount;
    uint32_t timestampValidBits;
    float timestampPeriodNanoseconds;
} PmQueueFamilyProperties;
PmResult pmGetPhysicalDeviceQueueFamilies(
    PmPhysicalDevice physicalDevice,
    uint32_t* pCount,
    PmQueueFamilyProperties* pProperties);
```

Supported vs enabled features [4.4]

```
typedef struct PmPhysicalDeviceFeatures {
    PmBool32 rigidBodies;
    PmBool32 rigidConstraints;
    PmBool32 particleLiquids;
    PmBool32 liquidRigidCoupling;
    PmBool32 liquidLiquidCoupling;
    PmBool32 cloth;
    PmBool32 rope;
    PmBool32 softBodies;
    PmBool32 smoke;
    PmBool32 domainCouplings;
    PmBool32 particleEmitters;
    PmBool32 paint;
    PmBool32 contactEvents;
    PmBool32 debugData;
    PmBool32 timelineSemaphores;
    PmBool32 timestampQueries;
    PmBool32 statisticsQueries;
    PmBool32 memoryBudget;
    PmBool32 vulkanInterop;
} PmPhysicalDeviceFeatures;
void pmGetPhysicalDeviceFeatures(
    PmPhysicalDevice physicalDevice,
    PmPhysicalDeviceFeatures* pFeatures);
```

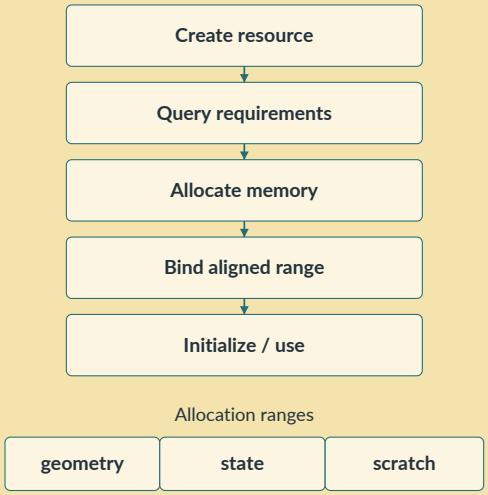
Execution profile negotiation [4.5]

```
#define
  PM_EXT_SIMULATION_PROFILE_EXTENSION_NAME
  "PM_EXT_simulation_profile"
#define
  PM_EXT_SIMULATION_PROFILE_SPEC_VERSION 1u
#define PM_EXECUTION_PROFILE_NAME_MAX_SIZE
  64u
#define
  PM_EXECUTION_PROFILE_REALTIME_SINGLE_STEP
  "realtime-single-step"
typedef struct PmExecutionProfileProperties
  {
    char
profileName[PM_EXECUTION_PROFILE_NAME_MAX_SIZE];
    uint32_t versionMajor;
    uint32_t versionMinor;
} PmExecutionProfileProperties;
typedef struct PmSimulationProfileCreateInfo
  {
    PmStructureType sType;
    const void* pNext;
    const char* pProfileName;
    uint32_t versionMajor;
    uint32_t versionMinor;
} PmSimulationProfileCreateInfo;
PmResult pmEnumerateExecutionProfiles(
    PmPhysicalDevice physicalDevice,
      uint32_t* pCount,
    PmExecutionProfileProperties*
      pProperties);
```

Logical device [4.6]

```
typedef struct PmDeviceQueueCreateInfo {
    uint32_t queueFamilyIndex;
    uint32_t queueCount;
    const float* pPriorities;
} PmDeviceQueueCreateInfo;
typedef struct PmDeviceCreateInfo {
    PmStructureType sType;
    const void* pNext;
    uint32_t queueCreateInfoCount;
    const PmDeviceQueueCreateInfo*
      pQueueInfos;
    PmPhysicalDeviceFeatures
      enabledFeatures;
    uint32_t enabledExtensionCount;
    const char* const*
      ppEnabledExtensionNames;
} PmDeviceCreateInfo;
PmResult pmCreateDevice(
    PmPhysicalDevice physicalDevice,
    const PmDeviceCreateInfo* pInfo,
    PmDevice* pDevice);
void pmDestroyDevice(PmDevice device);
void pmGetDeviceQueue(PmDevice device,
    uint32_t family, uint32_t index,
    PmQueue* pQueue);
```

Explicit device memory [5]



Heaps & types [5.1]

```
enum {
    PM_MEMORY_DEVICE_LOCAL_BIT = 0x1,
    PM_MEMORY_HOST_VISIBLE_BIT = 0x2,
    PM_MEMORY_HOST_COHERENT_BIT = 0x4,
    PM_MEMORY_HOST_CACHED_BIT = 0x8
};
typedef struct PmMemoryHeap {
    PmDeviceSize size;
    PmFlags flags;
} PmMemoryHeap;
typedef struct PmMemoryType {
    PmFlags propertyFlags;
    uint32_t heapIndex;
} PmMemoryType;
typedef struct PmMemoryProperties {
    uint32_t heapCount;
    PmMemoryHeap heaps[16];
    uint32_t typeCount;
    PmMemoryType types[32];
} PmMemoryProperties;
void pmGetPhysicalDeviceMemoryProperties(
    PmPhysicalDevice physicalDevice,
    PmMemoryProperties* pProperties);
```

Requirements & suballocation [5.2]

```
typedef struct PmMemoryRequirements {
    PmDeviceSize size;
    PmDeviceSize alignment;
    uint32_t memoryTypeBits;
    PmBool32 requiresDedicatedAllocation;
} PmMemoryRequirements;
```

Allocate & free [5.3]

```
typedef struct PmMemoryAllocateInfo {
    PmStructureType sType;
    const void* pNext;
    PmDeviceSize allocationSize;
    uint32_t memoryTypeIndex;
} PmMemoryAllocateInfo;
PmResult pmAllocateMemory(PmDevice device,
    const PmMemoryAllocateInfo* pInfo,
    PmDeviceMemory* pMemory);
```

Allocate & free (continued) [5.3]

```
void pmFreeMemory(PmDevice device,
    PmDeviceMemory memory);
```

Map & host visibility [5.4]

```
typedef struct PmMappedMemoryRange {
    PmDeviceMemory memory;
    PmDeviceSize offset;
    PmDeviceSize size;
} PmMappedMemoryRange;
PmResult pmMapMemory(PmDevice device,
    PmDeviceMemory memory,
    PmDeviceSize offset, PmDeviceSize size,
    void** ppData);
void pmUnmapMemory(PmDevice device,
    PmDeviceMemory memory);
PmResult pmFlushMappedMemoryRanges(
    PmDevice device, uint32_t count,
    const PmMappedMemoryRange* pRanges);
PmResult pmInvalidateMappedMemoryRanges(
    PmDevice device, uint32_t count,
    const PmMappedMemoryRange* pRanges);
```

Buffers & triangle geometry [6]

Buffer creation [6.1]

```
enum {
    PM_BUFFER_TRANSFER_SRC_BIT = 0x1,
    PM_BUFFER_TRANSFER_DST_BIT = 0x2,
    PM_BUFFER_GEOMETRY_INPUT_BIT = 0x4,
    PM_BUFFER_PARTICLE_INPUT_BIT = 0x8,
    PM_BUFFER_SIMULATION_OUTPUT_BIT = 0x10
};
typedef struct PmBufferCreateInfo {
    PmStructureType sType;
    const void* pNext;
    PmDeviceSize size;
    PmFlags usage;
    uint32_t initialQueueFamilyIndex;
} PmBufferCreateInfo;
PmResult pmCreateBuffer(PmDevice device,
    const PmBufferCreateInfo* pInfo,
    PmBuffer* pBuffer);
void pmDestroyBuffer(PmDevice device,
    PmBuffer buffer);
```

Buffer requirements & binding [6.2]

```
typedef struct PmBufferRange {
    PmBuffer buffer;
    PmDeviceSize offset;
    PmDeviceSize size;
} PmBufferRange;
void pmGetBufferMemoryRequirements(
    PmDevice device, PmBuffer buffer,
    PmMemoryRequirements* pRequirements);
PmResult pmBindBufferMemory(PmDevice device,
    PmBuffer buffer, PmDeviceMemory memory,
    PmDeviceSize offset);
```

Element formats [6.3]

```
typedef enum PmFormat {
    PM_FORMAT_UNDEFINED = 0,
    PM_FORMAT_R32_UINT = 1,
    PM_FORMAT_R32_SFLOAT = 2,
    PM_FORMAT_R32G32B32_SFLOAT = 3,
    PM_FORMAT_R32G32B32A32_SFLOAT = 4,
    PM_FORMAT_PARTICLE_STATE = 5,
    PM_FORMAT_RIGID_STATE = 6,
    PM_FORMAT_FLUID_PARTICLE_OUTPUT = 7
} PmFormat;
typedef struct PmBufferView {
    PmBufferRange range;
    PmFormat format;
    uint32_t count;
    uint32_t stride;
} PmBufferView;
```

Triangle geometry [6.4]

```
typedef struct PmGeometryCreateInfo {
    PmStructureType sType;
    const void* pNext;
    PmBufferView vertices;
    PmBufferView indices;
    uint32_t queueFamilyIndex;
} PmGeometryCreateInfo;
PmResult pmCreateGeometry(PmDevice device,
    const PmGeometryCreateInfo* pInfo,
    PmGeometry* pGeometry);
void pmDestroyGeometry(PmDevice device,
    PmGeometry geometry);
void pmGetGeometryMemoryRequirements(
    PmDevice device, PmGeometry geometry,
    PmMemoryRequirements* pRequirements);
PmResult pmBindGeometryMemory(
    PmDevice device, PmGeometry geometry,
```

Triangle geometry (continued) [6.4]

```
    PmDeviceMemory memory,
    PmDeviceSize offset);
```

Recorded geometry build [6.5]

```
PmResult pmCmdBuildGeometry(
    PmCommandBuffer commandBuffer,
    PmGeometry geometry);
```

Simulation state & rigid bodies [7]

Capacity-based simulation [7.1]

```
typedef struct PmFluidCapacity {
    uint32_t particleCapacity;
    uint32_t neighborCapacityPerParticle;
} PmFluidCapacity;
typedef struct PmClothCapacity {
    uint32_t vertexCapacity;
    uint32_t triangleIndexCapacity;
    uint32_t bondCapacity;
} PmClothCapacity;
typedef struct PmRopeCapacity {
    uint32_t nodeCapacity;
} PmRopeCapacity;
typedef struct PmSoftBodyCapacity {
    uint32_t nodeCapacity;
    uint32_t bondCapacity;
    uint32_t surfaceVertexCapacity;
    uint32_t surfaceTriangleIndexCapacity;
} PmSoftBodyCapacity;
typedef struct PmSmokeCapacity {
    uint32_t particleCapacity;
    uint32_t gridCellCapacity;
} PmSmokeCapacity;
typedef struct PmPaintFieldCapacity {
    uint32_t vertexCapacity;
    uint32_t width;
    uint32_t height;
} PmPaintFieldCapacity;
typedef struct PmSimulationCreateInfo {
    PmStructureType sType;
    const void* pNext;
    uint32_t queueFamilyIndex;
    uint32_t rigidBodyCapacity;
    uint32_t constraintCapacity;
    uint32_t contactCapacity;
    uint32_t fluidCount;
    const PmFluidCapacity* pFluidCapacities;
    uint32_t clothCount;
    const PmClothCapacity* pClothCapacities;
    uint32_t ropeCount;
    const PmRopeCapacity* pRopeCapacities;
    uint32_t softBodyCount;
    const PmSoftBodyCapacity*
      pSoftBodyCapacities;
    uint32_t smokeCount;
    const PmSmokeCapacity* pSmokeCapacities;
    uint32_t couplingCapacity;
    uint32_t emitterCapacity;
    uint32_t destroyerCapacity;
    uint32_t paintFieldCount;
    const PmPaintFieldCapacity*
      pPaintFieldCapacities;
    uint32_t paintRuleCapacity;
    uint32_t debugFrameCapacity;
} PmSimulationCreateInfo;
PmResult pmCreateSimulation(PmDevice device,
    const PmSimulationCreateInfo* pInfo,
    PmSimulation* pSimulation);
void pmDestroySimulation(PmDevice device,
    PmSimulation simulation);
```

Persistent & scratch bindings [7.2]

```
typedef struct
  PmSimulationMemoryRequirements {
    PmMemoryRequirements persistent;
    PmMemoryRequirements scratch;
} PmSimulationMemoryRequirements;
typedef struct PmSimulationMemoryBinding {
    PmDeviceMemory persistentMemory;
    PmDeviceSize persistentOffset;
    PmDeviceMemory scratchMemory;
    PmDeviceSize scratchOffset;
} PmSimulationMemoryBinding;
void pmGetSimulationMemoryRequirements(
    PmDevice device, PmSimulation
      simulation,
    PmSimulationMemoryRequirements* pInfo);
PmResult pmBindSimulationMemory(
    PmDevice device, PmSimulation
      simulation,
    const PmSimulationMemoryBinding*
      pBinding);
```

Portable rigid state record [7.3]

```
typedef struct PmRigidState {
    float position[3];
    float orientation[4];
    float linearVelocity[3];
    float angularVelocity[3];
} PmRigidState;
typedef enum PmMotionType {
    PM_MOTION_STATIC = 0,
    PM_MOTION_KINEMATIC = 1,
    PM_MOTION_DYNAMIC = 2
} PmMotionType;
```

Body descriptions [7.4]

```
typedef struct PmRigidBodyInfo {
    PmBool32 active;
    PmMotionType motionType;
    PmGeometry geometry;
    PmRigidState state;
    float mass;
    float inertiaDiagonal[3];
    float friction;
    float restitution;
    float linearDamping;
    float angularDamping;
    float maximumLinearSpeed;
    float maximumAngularSpeed;
    float collisionMargin;
    uint64_t userData;
} PmRigidBodyInfo;
PmResult pmCmdWriteRigidBodies(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t firstBody, uint32_t count,
    const PmRigidBodyInfo* pBodies);
```

Forces, impulses & targets [7.5]

```
typedef struct PmRigidInput {
    uint32_t bodyIndex;
    float force[3];
    float torque[3];
    float linearImpulse[3];
    float angularImpulse[3];
} PmRigidInput;
PmResult pmCmdApplyRigidInputs(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t count, const PmRigidInput*
      pInputs);
PmResult pmCmdSetRigidStates(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t firstBody, uint32_t count,
    const PmRigidState* pStates);
PmResult pmCmdSetKinematicTargets(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t firstBody, uint32_t count,
    const PmRigidState* pTargets);
```

Hit-box queries [7.6]

```
typedef struct PmHitBox {
    float center[3];
    float orientation[4];
    float halfExtents[3];
} PmHitBox;
typedef struct PmHitBoxParticle {
    uint32_t fluidIndex;
    uint32_t stableParticleId;
} PmHitBoxParticle;
typedef struct PmHitBoxOutputHeader {
    uint32_t rigidBodyCount;
    uint32_t particleCount;
    uint64_t revision;
} PmHitBoxOutputHeader;
PmResult pmCmdQueryHitBox(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    const PmHitBox* pBox,
    const PmBufferRange* pDestination);
```

Rigid constraints [8]

Joint families [8.1]

```
typedef enum PmConstraintType {
    PM_CONSTRAINT_FIXED = 0,
    PM_CONSTRAINT_POINT = 1,
    PM_CONSTRAINT_HINGE = 2,
    PM_CONSTRAINT_SLIDER = 3,
    PM_CONSTRAINT_PISTON = 4,
    PM_CONSTRAINT_GENERIC = 5,
    PM_CONSTRAINT_GENERIC_SPRING = 6,
    PM_CONSTRAINT_MOTOR = 7
} PmConstraintType;
typedef struct PmConstraintFrame {
    float position[3];
```

Joint families (continued) [8.1]

```
    float orientation[4];
} PmConstraintFrame;
```

Per-axis controls [8.2]

```
typedef struct PmAxisLimit {
    PmBool32 enabled;
    float lower;
    float upper;
} PmAxisLimit;
typedef struct PmAxisSpring {
    PmBool32 enabled;
    float stiffness;
    float damping;
    float equilibrium;
} PmAxisSpring;
typedef struct PmAxisMotor {
    PmBool32 enabled;
    float targetVelocity;
    float maximumForce;
} PmAxisMotor;
```

Constraint description [8.3]

```
typedef struct PmRigidConstraintInfo {
    PmBool32 active;
    PmConstraintType type;
    uint32_t bodyA;
    uint32_t bodyB;
    PmConstraintFrame frameA;
    PmConstraintFrame frameB;
    PmAxisLimit linearLimits[3];
    PmAxisLimit angularLimits[3];
    PmAxisSpring linearSprings[3];
    PmAxisSpring angularSprings[3];
    PmAxisMotor linearMotors[3];
    PmAxisMotor angularMotors[3];
    PmBool32 collideConnected;
    float breakingImpulseThreshold;
    uint32_t solverIterations;
} PmRigidConstraintInfo;
PmResult pmCmdWriteRigidConstraints(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t firstConstraint, uint32_t
      count,
    const PmRigidConstraintInfo*
      pConstraints);
```

Degrees of freedom [8.4]

FIXED	All translation and rotation locked
POINT	Three translations locked; rotation free
HINGE	Only local X rotation eligible
SLIDER	Only local X translation eligible
PISTON	Local X translation and rotation eligible
GENERIC	All six axes eligible for limits/motors
GENERIC_SPRING	All six axes; limits, springs and motors
MOTOR	All axes free except driven/limited axes

Physics pipelines & materials [9]

Rigid solver state [9.1]

```
typedef struct PmRigidSolverState {
    uint32_t rigidIterations;
    PmBool32 enableSleeping;
    float sleepLinearThreshold;
    float sleepAngularThreshold;
    float sleepTimeSeconds;
} PmRigidSolverState;
```

Particle-liquid material [9.2]

```
typedef struct PmLiquidMaterialState {
    float restDensity;
    float particleRadius;
    float supportRadius;
    float restParticleVolume;
    float pressureStiffness;
    float kinematicViscosity;
    float normalDamping;
    float velocityDamping;
    float maximumSpeed;
    float maximumPairAcceleration;
    uint32_t densityIterations;
} PmLiquidMaterialState;
```

Coupling state [9.3]

```
typedef enum PmInteractionFamily {
    PM_INTERACTION_FAMILY_FLUID = 0,
    PM_INTERACTION_FAMILY_CLOTH = 1,
    PM_INTERACTION_FAMILY_SOFT_BODY = 2,
    PM_INTERACTION_FAMILY_ROPE = 3,
    PM_INTERACTION_FAMILY_SMOKE = 4,
    PM_INTERACTION_FAMILY_RIGID_BODY = 5
} PmInteractionFamily;
typedef enum PmInteractionScope {
    PM_INTERACTION_SCOPE_SELF = 0,
    PM_INTERACTION_SCOPE_SAME_DOMAIN = 1,
    PM_INTERACTION_SCOPE_SEPARATE_SYSTEM =
      2,
    PM_INTERACTION_SCOPE_DISTINCT_FAMILY = 3
} PmInteractionScope;
typedef enum PmInteractionMode {
    PM_INTERACTION_MODE_CONTACT = 0,
    PM_INTERACTION_MODE_ATTACHMENT = 1,
    PM_INTERACTION_MODE_DRAG = 2,
    PM_INTERACTION_MODE_CONTAINMENT = 3,
    PM_INTERACTION_MODE_PRESSURE = 4,
    PM_INTERACTION_MODE_VISCOSITY = 5,
    PM_INTERACTION_MODE_THERMAL = 6,
    PM_INTERACTION_MODE_PHASE = 7,
    PM_INTERACTION_MODE_INTERFACE = 8
} PmInteractionMode;
typedef PmFlags PmInteractionModeFlags;
enum {
    PM_INTERACTION_MODE_CONTACT_BIT = 0x01,
    PM_INTERACTION_MODE_ATTACHMENT_BIT =
      0x02,
    PM_INTERACTION_MODE_DRAG_BIT = 0x04,
    PM_INTERACTION_MODE_CONTAINMENT_BIT =
      0x08,
    PM_INTERACTION_MODE_PRESSURE_BIT = 0x10,
    PM_INTERACTION_MODE_VISCOSITY_BIT =
      0x20,
    PM_INTERACTION_MODE_THERMAL_BIT = 0x40,
    PM_INTERACTION_MODE_PHASE_BIT = 0x80,
    PM_INTERACTION_MODE_INTERFACE_BIT =
      0x100
};
typedef enum PmInteractionDirection {
    PM_INTERACTION_DIRECTION_BIDIRECTIONAL =
      0,
    PM_INTERACTION_DIRECTION_FIRST_TO_SECOND
    = 1,
    PM_INTERACTION_DIRECTION_SECOND_TO_FIRST
    = 2,
PM_INTERACTION_DIRECTION_FIRST_TO_SECOND_TRANSFER
= 3,
PM_INTERACTION_DIRECTION_SECOND_TO_FIRST_TRANSFER
= 4,
PM_INTERACTION_DIRECTION_EXTERNAL_TO_FIRST =
  5,
PM_INTERACTION_DIRECTION_EXTERNAL_TO_SECOND
 = 6
} PmInteractionDirection;
typedef enum PmInteractionMaterialProfile {
    PM_INTERACTION_MATERIAL_ENDPOINT_NATIVE
     = 0,
    PM_INTERACTION_MATERIAL_SMOKE_PARTICLE =
      1,
    PM_INTERACTION_MATERIAL_SMOKE_GRID = 2,
    PM_INTERACTION_MATERIAL_EQUAL = 3,
    PM_INTERACTION_MATERIAL_MISCIBLE = 4,
PM_INTERACTION_MATERIAL_IMMISCIBLE_INTERFACE
= 5,
    PM_INTERACTION_MATERIAL_THERMAL_PHASE =
      6,
PM_INTERACTION_MATERIAL_SMOKE_PARTICLE_GRID
 = 7
} PmInteractionMaterialProfile;
typedef enum PmCouplingType {
    PM_COUPLING_LIQUID_RIGID = 0,
    PM_COUPLING_LIQUID_LIQUID = 1,
    PM_COUPLING_LIQUID_CLOTH = 2,
    PM_COUPLING_LIQUID_ROPE = 3,
    PM_COUPLING_LIQUID_SOFT_BODY = 4,
    PM_COUPLING_LIQUID_SMOKE = 5,
    PM_COUPLING_SOFT_BODY_CLOTH = 6,
    PM_COUPLING_ROPE_SOFT_BODY = 7,
    PM_COUPLING_ROPE_CLOTH = 8,
    PM_COUPLING_SMOKE_RIGID = 9,
    PM_COUPLING_SMOKE_CLOTH = 10,
    PM_COUPLING_SMOKE_ROPE = 11,
    PM_COUPLING_SMOKE_SOFT_BODY = 12,
    PM_COUPLING_SMOKE_SMOKE = 13,
    PM_COUPLING_SOFT_BODY_SOFT_BODY = 14
} PmCouplingType;
typedef struct PmMechanicalCouplingState {
    float contactDistance;
    float stiffness;
    float damping;
    float friction;
    float tangentialDrag;
    float maximumForce;
    float maximumAcceleration;
    PmBool32 twoWay;
```

Coupling state (continued) [9.3]

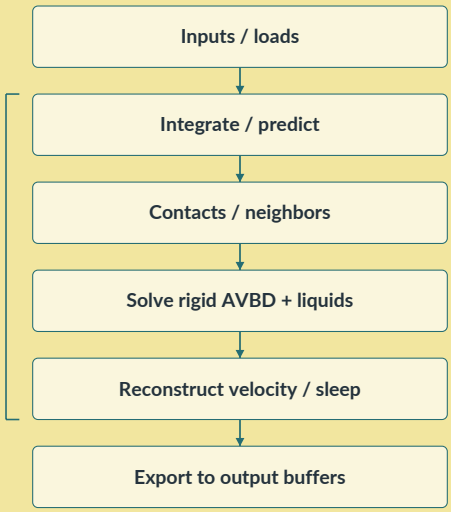
```
} PmMechanicalCouplingState;
typedef struct PmSmokeCouplingState {
    float windDrag;
    float maximumWindAcceleration;
    float airDensity;
    float dragCoefficient;
    float contactDistance;
    PmBool32 tracerContact;
} PmSmokeCouplingState;
typedef struct PmThermalCouplingState {
    float heaterCenter[3];
    float heaterOrientation[4];
    float heaterHalfExtents[2];
    float heaterTemperature;
    float boilingTemperature;
    float heatTransferRate;
    float windDrag;
    float steamRiseSpeed;
} PmThermalCouplingState;
typedef struct PmAttachmentCouplingState {
    uint32_t firstVertex;
    uint32_t lastVertex;
    float effectiveMass;
    float supportRadiusScale;
    float contactSupportRadiusScale;
    float maximumAcceleration;
    PmBool32 attachFirst;
    PmBool32 attachLast;
} PmAttachmentCouplingState;
typedef struct PmCouplingParameters {
    PmMechanicalCouplingState mechanical;
    PmSmokeCouplingState smoke;
    PmAttachmentCouplingState attachment;
    PmThermalCouplingState thermal;
} PmCouplingParameters;
typedef struct PmCouplingState {
    PmCouplingType type;
    uint32_t first, second;
    PmInteractionModeFlags modeFlags;
    PmInteractionMaterialProfile
      materialProfile;
    PmCouplingParameters parameters;
} PmCouplingState;
typedef struct PmCouplingStateInfo {
    PmStructureType sType;
    const void* pNext;
    PmCouplingType type;
    uint32_t first, second;
    PmInteractionModeFlags modeFlags;
    PmInteractionMaterialProfile
      materialProfile;
    PmCouplingParameters parameters;
} PmCouplingStateInfo;
typedef struct PmFluidPairMaterialState {
    PmStructureType sType;
    const void* pNext;
    uint32_t firstMaterialIndex;
    uint32_t secondMaterialIndex;
} PmFluidPairMaterialState;
typedef struct PmFluidInterfaceState {
    PmStructureType sType;
    const void* pNext;
    float surfaceTension;
} PmFluidInterfaceState;
PmResult pmCmdSetCouplings(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      couplingCount,
    const PmCouplingState *pCouplings);
PmResult pmCmdSetCouplings2(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      couplingCount,
    const PmCouplingStateInfo *pCouplings);
```

Deformable, rope & smoke materials [9.4]

```
typedef struct PmClothMaterialState {
    float vertexMass;
    float thickness;
    float stretchCompliance;
    float bendingCompliance;
    float velocityDamping;
    float contactFriction;
    uint32_t solverIterations;
    float breakStrain;
    uint32_t fracturePersistenceSubsteps;
    float impactBreakImpulse;
    PmBool32 preserveVolume;
    float targetVolume;
    float volumeCompliance;
} PmClothMaterialState;
typedef struct PmRopeMaterialState {
    float nodeSpacing;
    float radius;
    float mass;
    float stretchCompliance;
    float velocityDamping;
    float maximumSubstepTimestep;
    float friction;
```

Deformable, rope & smoke materials (continued) [9.4]

```
    float maximumSpeed;
    uint32_t solverIterations;
    PmBool32 selfCollision;
} PmRopeMaterialState;
typedef struct PmSoftBodyMaterialState {
    float nodeMass;
    float nodeRadius;
    float stretchCompliance;
    float velocityDamping;
    float springDamping;
    float contactFriction;
    float shapeMatchingStiffness;
    float maximumProjectionFraction;
    float constraintVelocityResponse;
    float maximumSpeed;
    uint32_t solverIterations;
} PmSoftBodyMaterialState;
typedef struct PmSmokeMaterialState {
    float particleRadius;
    float buoyancy;
    float response;
    float restNumberDensity;
    float pressureStiffness;
    float viscosity;
    float vorticityConfinement;
    float maximumSpeed;
    uint32_t gridResolution;
    uint32_t gridVerticalResolution;
    uint32_t gridPressureIterations;
    float gridKinematicViscosity;
    float gridLesCoefficient;
    float gridPressureTolerance;
} PmSmokeMaterialState;
```



Substep loop

Pipeline creation [9.5]

```
typedef struct
  PmPhysicsPipelineCouplingStateInfo {
    PmStructureType sType;
    const void* pNext;
    uint32_t couplingCount;
    const PmCouplingStateInfo* pCouplings;
} PmPhysicsPipelineCouplingStateInfo;
typedef struct PmPhysicsPipelineCreateInfo {
    PmStructureType sType;
    const void* pNext;
    const PmRigidSolverState* pRigidState;
    uint32_t fluidMaterialCount;
    const PmLiquidMaterialState*
      pFluidMaterials;
    uint32_t couplingCount;
    const PmCouplingState* pCouplings;
    uint32_t clothMaterialCount;
    const PmClothMaterialState*
      pClothMaterials;
    uint32_t ropeMaterialCount;
    const PmRopeMaterialState*
      pRopeMaterials;
    uint32_t softBodyMaterialCount;
    const PmSoftBodyMaterialState*
      pSoftBodyMaterials;
    uint32_t smokeMaterialCount;
    const PmSmokeMaterialState*
      pSmokeMaterials;
} PmPhysicsPipelineCreateInfo;
PmResult pmCreatePhysicsPipeline(
    PmDevice device,
    const PmPhysicsPipelineCreateInfo*
```

Pipeline creation (continued) [9.5]

```
      pInfo,
    PmPhysicsPipeline* pPipeline);
void pmDestroyPhysicsPipeline(
    PmDevice device,
    PmPhysicsPipeline pipeline);
```

Simulation domains & events [10]

Shared records [10.1]

```
typedef struct PmFloat2 { float x; float y;
  } PmFloat2;
typedef struct PmFloat3 { float x; float y;
  float z; } PmFloat3;
typedef struct PmSoftBodyBond {
    uint32_t first;
    uint32_t second;
    float restLength;
} PmSoftBodyBond;
typedef struct PmSoftBodySurfaceBinding {
    uint32_t nodes[4];
    float weights[4];
} PmSoftBodySurfaceBinding;
typedef struct PmDomainOutputHeader {
    uint32_t activeCount;
    uint32_t capacity;
    uint64_t revision;
} PmDomainOutputHeader;
```

Cloth resources [10.2]

```
typedef struct PmClothInfo {
    PmBool32 active;
    uint32_t materialIndex;
    uint32_t vertexCount;
    const PmFloat3* pVertices;
    uint32_t triangleIndexCount;
    const uint32_t* pTriangleIndices;
    const float* pInverseMasses;
} PmClothInfo;
typedef struct PmClothVertexState {
    PmFloat3 position;
    PmFloat3 velocity;
    PmFloat3 rigidContactForce;
    PmFloat3 liquidContactForce;
    PmFloat3 softBodyContactForce;
    PmFloat3 ropeContactForce;
} PmClothVertexState;
PmResult pmCmdWriteCloths(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstCloth,
    uint32_t count, const PmClothInfo*
      pCloths);
PmResult pmCmdCopyClothState(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      clothIndex,
    const PmBufferRange* pDestination);
PmResult
  pmCmdCopyClothSurface(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      clothIndex,
    const PmBufferRange* pDestination);
```

Rope resources [10.3]

```
typedef struct PmRopeAttachment {
    uint32_t rigidBodyIndex;
    PmFloat3 localAnchor;
    PmBool32 enabled;
} PmRopeAttachment;
typedef struct PmRopeInfo {
    PmBool32 active;
    uint32_t materialIndex;
    uint32_t pointCount;
    const PmFloat3* pCenterline;
    PmRopeAttachment first;
    PmRopeAttachment last;
} PmRopeInfo;
typedef struct PmRopeNodeState {
    PmFloat3 position;
    PmFloat3 velocity;
    PmFloat3 constraintForce;
    PmFloat3 contactForce;
    PmFloat3 liquidContactForce;
    PmFloat3 softBodyContactForce;
} PmRopeNodeState;
PmResult pmCmdWriteRopes(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstRope,
    uint32_t count, const PmRopeInfo*
      pRopes);
PmResult pmCmdCopyRopeState(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
```

Rope resources (continued) [10.3]

```
      ropeIndex,
    const PmBufferRange* pDestination);
```

Soft-body resources [10.4]

```
typedef struct PmSoftBodyInfo {
    PmBool32 active;
    uint32_t materialIndex;
    uint32_t nodeCount;
    const PmFloat3* pNodes;
    uint32_t bondCount;
    const PmSoftBodyBond* pBonds;
    const float* pInverseMasses;
    uint32_t surfaceVertexCount;
    const PmFloat3* pSurfaceVertices;
    uint32_t surfaceTriangleIndexCount;
    const uint32_t* pSurfaceTriangleIndices;
    const PmSoftBodySurfaceBinding*
      pSurfaceBindings;
} PmSoftBodyInfo;
typedef struct PmSoftBodyStateInfo {
    PmStructureType sType;
    const void* pNext;
    PmBool32 active;
    uint32_t materialIndex;
    uint32_t nodeCount;
    const PmFloat3* pNodes;
    uint32_t bondCount;
    const PmSoftBodyBond* pBonds;
    const float* pInverseMasses;
    uint32_t surfaceVertexCount;
    const PmFloat3* pSurfaceVertices;
    uint32_t surfaceTriangleIndexCount;
    const uint32_t* pSurfaceTriangleIndices;
    const PmSoftBodySurfaceBinding*
      pSurfaceBindings;
} PmSoftBodyStateInfo;
typedef struct PmSoftBodySelfContactState {
    PmStructureType sType;
    const void* pNext;
    PmBool32 enabled;
} PmSoftBodySelfContactState;
typedef struct PmSoftBodyNodeState {
    PmFloat3 position;
    PmFloat3 velocity;
    PmFloat3 rigidContactForce;
    PmFloat3 clothContactForce;
    PmFloat3 liquidContactForce;
    PmFloat3 ropeContactForce;
} PmSoftBodyNodeState;
PmResult
  pmCmdWriteSoftBodies(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstSoftBody,
    uint32_t count, const PmSoftBodyInfo*
      pSoftBodies);
PmResult
  pmCmdWriteSoftBodies2(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstSoftBody,
    uint32_t count, const
      PmSoftBodyStateInfo* pSoftBodies);
PmResult
  pmCmdCopySoftBodyState(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      softBodyIndex,
    const PmBufferRange* pDestination);
PmResult
  pmCmdCopySoftBodySurface(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      softBodyIndex,
    const PmBufferRange* pDestination);
```

Smoke resources [10.5]

```
typedef struct PmSmokeInfo {
    PmBool32 active;
    uint32_t materialIndex;
    uint32_t particleCapacity;
    PmFloat3 emitterCenter;
    PmFloat2 emitterHalfExtents;
    PmFloat3 initialVelocity;
    PmFloat3 wind;
    float particlesPerSecond;
    float lifetime;
    PmFloat3 gridMinimum;
    float gridEdgeLength;
} PmSmokeInfo;
typedef struct PmSmokeParticleState {
    PmFloat3 position;
    PmFloat3 velocity;
    float age;
    float numberDensity;
    float pressure;
    PmFloat3 vorticity;
} PmSmokeParticleState;
```

Smoke resources (continued) [10.5]

```
PmResult pmCmdWriteSmoke(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstSmoke,
    uint32_t count, const PmSmokeInfo*
      pSmoke);
PmResult pmCmdCopySmokeState(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      smokeIndex,
    const PmBufferRange* pDestination);
```

Emitters & destruction [10.6]

```
typedef struct PmParticleEmitterInfo {
    PmBool32 active;
    uint32_t fluidIndex;
    uint32_t vertexCount;
    const PmFloat3* pVertices;
    uint32_t triangleIndexCount;
    const uint32_t* pTriangleIndices;
    float spacing;
    PmFloat3 initialVelocity;
    float initialTemperatureCelsius;
} PmParticleEmitterInfo;
typedef enum PmCrossingDirection {
    PM_CROSSING_ALONG_NORMAL = 0,
    PM_CROSSING_AGAINST_NORMAL = 1,
    PM_CROSSING_EITHER = 2
} PmCrossingDirection;
typedef struct PmParticleDestroyerInfo {
    PmBool32 active;
    uint32_t fluidIndex;
    PmFloat3 center;
    float orientation[4];
    PmFloat2 halfExtents;
    PmCrossingDirection crossing;
} PmParticleDestroyerInfo;
PmResult
  pmCmdWriteParticleEmitters(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstEmitter,
    uint32_t count, const
      PmParticleEmitterInfo* pEmitters);
PmResult
pmCmdWriteParticleDestroyers(PmCommandBuffer
commandBuffer,
    PmSimulation simulation, uint32_t
      firstDestroyer,
    uint32_t count, const
      PmParticleDestroyerInfo* pDestroyers);
```

Paint fields & rules [10.7]

```
typedef enum PmPaintTargetType {
    PM_PAINT_TARGET_RIGID = 0,
    PM_PAINT_TARGET_CLOTH = 1
} PmPaintTargetType;
typedef struct PmPaintFieldInfo {
    PmBool32 active;
    PmPaintTargetType targetType;
    uint32_t targetIndex;
    PmGeometry geometry;
    uint32_t vertexCount;
    const PmFloat2* pVertexUvs;
    uint32_t width;
    uint32_t height;
} PmPaintFieldInfo;
typedef enum PmPaintSourceType {
    PM_PAINT_SOURCE_LIQUID = 0,
    PM_PAINT_SOURCE_RIGID = 1
} PmPaintSourceType;
typedef struct PmPaintRuleInfo {
    PmBool32 active;
    PmPaintSourceType sourceType;
    uint32_t sourceIndex;
    uint32_t targetFieldIndex;
    float reach;
    float brushRadius;
} PmPaintRuleInfo;
PmResult
  pmCmdWritePaintFields(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstField,
    uint32_t count, const PmPaintFieldInfo*
      pFields);
PmResult
  pmCmdWritePaintRules(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstRule,
    uint32_t count, const PmPaintRuleInfo*
      pRules);
PmResult
  pmCmdClearPaintFields(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstField,
```

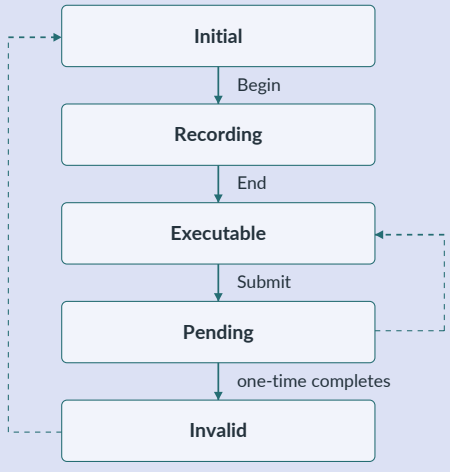
Paint fields & rules (continued) [10.7]

```
    uint32_t count);
PmResult pmCmdCopyPaintField(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      fieldIndex,
    const PmBufferRange* pDestination);
```

Constraint, contact & debug output [10.8]

```
typedef struct PmConstraintState {
    PmBool32 enabled;
    PmBool32 broken;
    float appliedImpulse;
} PmConstraintState;
typedef enum PmContactEventType {
    PM_CONTACT_RIGID_RIGID = 0,
    PM_CONTACT_LIQUID_RIGID = 1
} PmContactEventType;
typedef struct PmContactEvent {
    PmContactEventType type;
    uint32_t first;
    uint32_t second;
    PmFloat3 position;
    PmFloat3 normal;
    float penetration;
    float normalImpulse;
    PmFloat3 frictionImpulse;
} PmContactEvent;
typedef enum PmDebugRecordType {
    PM_DEBUG_RIGID = 0,
    PM_DEBUG_LIQUID = 1,
    PM_DEBUG_CLOTH = 2,
    PM_DEBUG_ROPE = 3,
    PM_DEBUG_SOFT_BODY = 4
} PmDebugRecordType;
typedef struct PmDebugRecord {
    PmDebugRecordType type;
    uint32_t objectIndex;
    uint32_t elementIndex;
    PmFloat3 position;
    PmFloat3 velocity;
    PmFloat3 force;
} PmDebugRecord;
PmResult
  pmCmdCopyConstraintStates(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, uint32_t
      firstConstraint,
    uint32_t count, const PmBufferRange*
      pDestination);
PmResult
  pmCmdCopyContactEvents(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, const
      PmBufferRange* pDestination);
PmResult pmCmdCopyDebugData(PmCommandBuffer
  commandBuffer,
    PmSimulation simulation, const
      PmBufferRange* pDestination);
```

Command recording & lifecycle [11]



Reusable completion → Executable

Reset → Initial; never while Pending

Command pools [11.1]

```
enum {
    PM_COMMAND_POOL_RESET_BUFFER_BIT = 0x1,
    PM_COMMAND_BUFFER_ONE_TIME_BIT = 0x1
};
typedef struct PmCommandPoolCreateInfo {
    PmStructureType sType;
    const void* pNext;
    PmFlags flags;
```

Command pools (continued) [11.1]

```
    uint32_t queueFamilyIndex;
} PmCommandPoolCreateInfo;
PmResult pmCreateCommandPool(PmDevice
  device,
    const PmCommandPoolCreateInfo* pInfo,
    PmCommandPool* pPool);
void pmDestroyCommandPool(PmDevice device,
    PmCommandPool pool);
PmResult pmResetCommandPool(PmDevice device,
    PmCommandPool pool);
```

Allocate, record & reset [11.2]

```
typedef struct PmCommandBufferBeginInfo {
    PmStructureType sType;
    const void* pNext;
    PmFlags flags;
} PmCommandBufferBeginInfo;
PmResult pmAllocateCommandBuffers(
    PmDevice device, PmCommandPool pool,
    uint32_t count, PmCommandBuffer*
      pBuffers);
void pmFreeCommandBuffers(PmDevice device,
    PmCommandPool pool, uint32_t count,
    const PmCommandBuffer* pBuffers);
PmResult pmBeginCommandBuffer(
    PmCommandBuffer commandBuffer,
    const PmCommandBufferBeginInfo* pInfo);
PmResult pmEndCommandBuffer(
    PmCommandBuffer commandBuffer);
PmResult pmResetCommandBuffer(
    PmCommandBuffer commandBuffer);
```

Legal states & resource lifetime [11.3]

Initial	Begin → Recording; may free/reset
Recording	pmCmd* only; End → Executable
Executable	QueueSubmit → Pending; may reset/free
Pending	No edits, reset, free or resubmit
Completion	Reusable → Executable; one-time → Invalid
Invalid	Begin → Recording; reset → Initial, or free

Reset & particle initialization [11.4]

```
typedef struct PmParticleState {
    float position[3];
    float velocity[3];
    float temperatureCelsius;
} PmParticleState;
typedef struct PmFluidParticleOutput {
    float position[3];
    float velocity[3];
    float acceleration[3];
    float temperatureCelsius;
    float foam;
    uint32_t stableId;
} PmFluidParticleOutput;
PmResult pmCmdResetSimulation(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation);
PmResult pmCmdSetFluidParticles(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t fluidIndex,
    const PmBufferView* pParticles);
```

Bind & step [11.5]

```
enum {
    PM_STEP_COLLECT_CONTACT_EVENTS_BIT = 0x1
};
typedef struct PmStepInfo {
    float timestep;
    uint32_t substeps;
    float gravity[3];
    PmFlags flags;
} PmStepInfo;
PmResult pmCmdBindPhysicsPipeline(
    PmCommandBuffer commandBuffer,
    PmPhysicsPipeline pipeline);
PmResult pmCmdStepSimulation(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    const PmStepInfo* pStep);
```

Data movement & output [12]

Buffer transfers [12.1]

```
typedef struct PmBufferCopy {
    PmDeviceSize srcOffset;
    PmDeviceSize dstOffset;
    PmDeviceSize size;
} PmBufferCopy;
PmResult pmCmdCopyBuffer(
    PmCommandBuffer commandBuffer,
    PmBuffer src, PmBuffer dst,
    uint32_t count,
    const PmBufferCopy* pRegions);
```

Copy simulation output [12.2]

```
PmResult pmCmdCopyRigidStates(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t firstBody, uint32_t count,
    const PmBufferRange* pDestination);
PmResult pmCmdCopyFluidParticles(
    PmCommandBuffer commandBuffer,
    PmSimulation simulation,
    uint32_t fluidIndex,
    const PmBufferRange* pDestination);
```

Submission & synchronization [13]

Stage and access scopes [13.1]

```
#define PM_STAGE_HOST_BIT UINT64_C(0x1)
#define PM_STAGE_TRANSFER_BIT UINT64_C(0x2)
#define PM_STAGE_UPDATE_BIT UINT64_C(0x4)
#define PM_STAGE_GEOMETRY_BUILD_BIT
  UINT64_C(0x8)
#define PM_STAGE_PHYSICS_BIT UINT64_C(0x10)
#define PM_STAGE_ALL_COMMANDS_BIT
  UINT64_C(0x1e)
#define PM_ACCESS_HOST_READ_BIT
  UINT64_C(0x1)
#define PM_ACCESS_HOST_WRITE_BIT
  UINT64_C(0x2)
#define PM_ACCESS_READ_BIT UINT64_C(0x4)
#define PM_ACCESS_WRITE_BIT UINT64_C(0x8)
```

Memory & buffer barriers [13.2]

```
typedef struct PmMemoryBarrier {
    PmStageFlags srcStages;
    PmAccessFlags srcAccess;
    PmStageFlags dstStages;
    PmAccessFlags dstAccess;
} PmMemoryBarrier;
typedef struct PmBufferMemoryBarrier {
    PmMemoryBarrier memory;
    PmBufferRange range;
    uint32_t srcQueueFamilyIndex;
    uint32_t dstQueueFamilyIndex;
} PmBufferMemoryBarrier;
PmResult pmCmdPipelineBarrier(
    PmCommandBuffer commandBuffer,
    uint32_t memoryBarrierCount,
    const PmMemoryBarrier* pMemoryBarriers,
    uint32_t bufferBarrierCount,
    const PmBufferMemoryBarrier*
      pBufferBarriers);
```

Timeline semaphores [13.3]

```
typedef struct PmSemaphoreCreateInfo {
    PmStructureType sType;
    const void* pNext;
    uint64_t initialValue;
} PmSemaphoreCreateInfo;
typedef struct PmSemaphoreSubmitInfo {
    PmSemaphore semaphore;
    uint64_t value;
    PmStageFlags stages;
} PmSemaphoreSubmitInfo;
PmResult pmCreateSemaphore(PmDevice device,
    const PmSemaphoreCreateInfo* pInfo,
    PmSemaphore* pSemaphore);
void pmDestroySemaphore(PmDevice device,
    PmSemaphore semaphore);
PmResult pmGetSemaphoreCounterValue(
    PmDevice device, PmSemaphore semaphore,
    uint64_t* pValue);
PmResult pmWaitSemaphore(PmDevice device,
    PmSemaphore semaphore, uint64_t value,
    uint64_t timeoutNanoseconds);
```

Queue submission [13.4]

```
typedef struct PmSubmitInfo {
    PmStructureType sType;
    const void* pNext;
    uint32_t waitCount;
    const PmSemaphoreSubmitInfo* pWaits;
    uint32_t commandBufferCount;
    const PmCommandBuffer* pCommandBuffers;
    uint32_t signalCount;
```

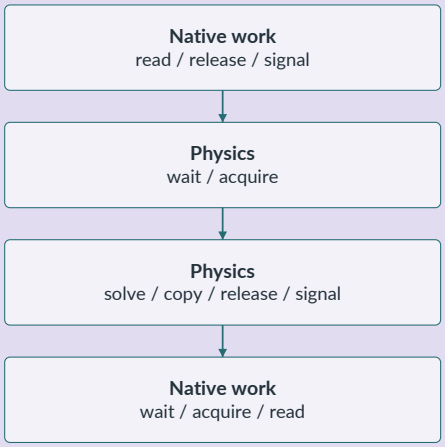
Queue submission (continued) [13.4]

```
    const PmSemaphoreSubmitInfo* pSignals;
} PmSubmitInfo;
PmResult pmQueueSubmit(PmQueue queue,
    const PmSubmitInfo* pInfo, PmFence
      fence);
PmResult pmQueueWaitIdle(PmQueue queue);
PmResult pmDeviceWaitIdle(PmDevice device);
```

Host completion fences [13.5]

```
enum { PM_FENCE_SIGNALED_BIT = 0x1 };
PmResult pmCreateFence(PmDevice device,
    PmFlags flags, PmFence* pFence);
void pmDestroyFence(PmDevice device,
    PmFence fence);
PmResult pmGetFenceStatus(PmDevice device,
    PmFence fence);
PmResult pmResetFences(PmDevice device,
    uint32_t count, const PmFence* pFences);
PmResult pmWaitForFences(PmDevice device,
    uint32_t count, const PmFence* pFences,
    PmBool32 waitAll, uint64_t
      timeoutNanoseconds);
```

Native API interoperability [14]



Timeline + resource handoff

Backend capabilities [14.1]

```
typedef enum PmNativeBackend {
    PM_NATIVE_BACKEND_CUDA = 1,
    PM_NATIVE_BACKEND_METAL = 2,
    PM_NATIVE_BACKEND_D3D12 = 3,
    PM_NATIVE_BACKEND_VULKAN = 4
} PmNativeBackend;
typedef uint64_t PmNativeHandle;
typedef struct PmNativeInteropProperties {
    PmBool32 deviceImport;
    PmBool32 bufferImport;
    PmBool32 timelineImport;
} PmNativeInteropProperties;
PmResult
  pmGetPhysicalDeviceInteropProperties(
    PmPhysicalDevice physicalDevice,
    PmNativeBackend backend,
    PmNativeInteropProperties* pProperties);
```

Device and queue import [14.2]

```
enum {
    PM_NATIVE_TIMELINE_ENABLED_BIT = 0x1,
    PM_NATIVE_SYNCHRONIZATION2_BIT = 0x2
};
typedef struct PmNativeQueueInfo {
    PmNativeHandle queue;
    uint32_t familyIndex;
    uint32_t queueIndex;
} PmNativeQueueInfo;
typedef struct PmNativeDeviceInfo {
    PmNativeBackend backend;
    PmNativeHandle instance;
    PmNativeHandle physicalDevice;
    PmNativeHandle device;
} PmNativeDeviceInfo;
typedef struct PmNativeDeviceImportInfo {
    PmStructureType sType;
    const void* pNext;
    PmNativeDeviceInfo nativeDevice;
    uint32_t queueCount;
    const PmNativeQueueInfo* pQueues;
    PmFlags enabledNativeFeatures;
    PmPhysicalDeviceFeatures
      enabledFeatures;
} PmNativeDeviceImportInfo;
```

Device and queue import (continued) [14.2]

```
PmResult pmImportNativeDevice(
    PmInstance instance,
    const PmNativeDeviceImportInfo* pInfo,
    PmDevice* pDevice);
PmResult pmGetNativeDevice(PmDevice device,
    PmNativeDeviceInfo* pNativeDevice);
PmResult pmGetNativeQueue(PmQueue queue,
    PmNativeQueueInfo* pNativeQueue);
```

Buffer import [14.3]

```
typedef struct PmNativeBufferInfo {
    PmNativeHandle buffer;
    PmDeviceSize offset;
    PmDeviceSize size;
    PmFlags usage;
    uint64_t nativeUsage;
    uint32_t owningQueueFamilyIndex;
} PmNativeBufferInfo;
PmResult pmImportNativeBuffer(PmDevice
  device,
    const PmNativeBufferInfo* pInfo,
    PmBuffer* pBuffer);
PmResult pmGetNativeBuffer(PmDevice device,
    PmBuffer buffer,
    PmNativeBufferInfo* pNativeBuffer);
```

Timeline import [14.4]

```
PmResult pmImportNativeSemaphore(PmDevice
  device,
    PmNativeHandle nativeSemaphore,
    PmSemaphore* pSemaphore);
PmResult pmGetNativeSemaphore(PmDevice
  device,
    PmSemaphore semaphore,
    PmNativeHandle* pNativeSemaphore);
```

Native object mapping [14.5]

CUDA	CUcontext / CUstream; CUdeviceptr; CUexternalSemaphore
Metal	MTLDevice / MTLCommandQueue; MTLBuffer; MTLSharedEvent
DirectX 12	ID3D12Device / CommandQueue; Resource; Fence
Vulkan	VkDevice / VkQueue; VkBuffer; timeline VkSemaphore

Typed Vulkan device adapter [14.6]

```
typedef struct PmVulkanQueueInfo {
    VkQueue queue;
    uint32_t familyIndex;
    uint32_t queueIndex;
} PmVulkanQueueInfo;
typedef struct PmVulkanDeviceInfo {
    VkInstance instance;
    VkPhysicalDevice physicalDevice;
    VkDevice device;
} PmVulkanDeviceInfo;
typedef struct PmVulkanDeviceImportInfo {
    VkInstance instance;
    VkPhysicalDevice physicalDevice;
    VkDevice device;
    uint32_t queueCount;
    const PmVulkanQueueInfo* pQueues;
    VkBool32 timelineSemaphoreEnabled;
    VkBool32 synchronization2Enabled;
    PmPhysicalDeviceFeatures
      enabledFeatures;
} PmVulkanDeviceImportInfo;
PmResult pmImportVulkanDeviceEXT(
    PmInstance instance,
    const PmVulkanDeviceImportInfo* pInfo,
    PmDevice* pDevice);
PmResult pmGetVulkanDeviceEXT(PmDevice
  device,
    PmVulkanDeviceInfo* pNativeDevice);
```

Typed Vulkan buffer adapter [14.7]

```
typedef struct PmVulkanBufferImportInfo {
    VkBuffer buffer;
    VkDeviceSize size;
    VkBufferUsageFlags nativeUsage;
    PmFlags usage;
    uint32_t owningQueueFamilyIndex;
} PmVulkanBufferImportInfo;
PmResult pmImportVulkanBufferEXT(
    PmDevice device,
    const PmVulkanBufferImportInfo* pInfo,
    PmBuffer* pBuffer);
PmResult pmGetVulkanBufferEXT(PmDevice
  device,
```

Typed Vulkan buffer adapter (continued) [14.7]

```
    PmBuffer buffer, VkBuffer*
      pNativeBuffer);
```

Typed Vulkan timeline adapter [14.8]

```
PmResult pmImportVulkanSemaphoreEXT(
    PmDevice device, VkSemaphore semaphore,
    PmSemaphore* pSemaphore);
PmResult pmGetVulkanSemaphoreEXT(
    PmDevice device, PmSemaphore semaphore,
    VkSemaphore* pNativeSemaphore);
```

Native synchronization [14.10]

CUDA	Stream order + external timeline waits/signals
Metal	Command buffers + shared events + resource hazards
DirectX 12	Queue fences + resource barriers
Vulkan	Timeline semaphores + memory/ownership barriers

Timing, work & memory queries [15]

Query pools [15.1]

```
typedef enum PmQueryType {
    PM_QUERY_TIMESTAMP = 0,
    PM_QUERY_STEP_STATISTICS = 1
} PmQueryType;
typedef struct PmQueryPoolCreateInfo {
    PmStructureType sType;
    const void* pNext;
    PmQueryType queryType;
    uint32_t queryCount;
} PmQueryPoolCreateInfo;
PmResult pmCreateQueryPool(PmDevice device,
    const PmQueryPoolCreateInfo* pInfo,
    PmQueryPool* pPool);
void pmDestroyQueryPool(PmDevice device,
    PmQueryPool pool);
PmResult pmCmdResetQueryPool(
    PmCommandBuffer commandBuffer,
    PmQueryPool pool, uint32_t first,
    uint32_t count);
```

GPU timestamps [15.2]

```
PmResult pmCmdWriteTimestamp(
    PmCommandBuffer commandBuffer,
    PmStageFlags stage,
    PmQueryPool pool, uint32_t query);
```

Step statistics & execution status [15.3]

```
typedef struct PmStepStatistics {
    uint64_t stepCount;
    uint64_t substepsExecuted;
    uint64_t poseIterations;
    uint64_t impactIterations;
    uint64_t peakContactCount;
    uint64_t peakActiveBodies;
    uint64_t peakAwakeBodies;
    uint64_t peakParticleCount;
    uint64_t scratchHighWaterBytes;
    uint64_t overflowCount;
    int32_t executionResult;
    uint32_t earlyExitCount;
} PmStepStatistics;
PmResult pmCmdBeginStatisticsQuery(
    PmCommandBuffer commandBuffer,
    PmQueryPool pool, uint32_t query);
PmResult pmCmdEndStatisticsQuery(
    PmCommandBuffer commandBuffer,
    PmQueryPool pool, uint32_t query);
PmResult pmGetSimulationStatus(
    PmDevice device, PmSimulation
      simulation,
    PmResult* pExecutionResult);
```

Readback & availability [15.4]

```
enum {
    PM_QUERY_RESULT_WAIT_BIT = 0x1,
    PM_QUERY_RESULT_WITH_AVAILABILITY_BIT =
      0x2
};
PmResult pmGetQueryPoolResults(PmDevice
  device,
    PmQueryPool pool, uint32_t first,
    uint32_t count, size_t dataSize,
    void* pData, PmDeviceSize stride,
    PmFlags flags);
```

Device budget & library accounting [15.5]

```
typedef enum PmMemoryBudgetConfidence {
    PM_MEMORY_BUDGET_CONFIDENCE_UNKNOWN = 0,
PM_MEMORY_BUDGET_CONFIDENCE_IMPLEMENTATION_ESTIMATE
= 1,
PM_MEMORY_BUDGET_CONFIDENCE_DRIVER_REPORTED
 = 2,
    PM_MEMORY_BUDGET_CONFIDENCE_OS_REPORTED
     = 3
} PmMemoryBudgetConfidence;
typedef struct PmMemoryBudget {
    uint32_t heapCount;
    PmDeviceSize budget[16];
    PmDeviceSize usage[16];
    PmMemoryBudgetConfidence confidence[16];
    PmDeviceSize applicationAllocationBytes;
    PmDeviceSize libraryDeviceBytes;
    PmDeviceSize libraryHostBytes;
    PmDeviceSize borrowedBufferBytes;
} PmMemoryBudget;
PmResult pmGetDeviceMemoryBudget(
    PmDevice device, PmMemoryBudget*
      pBudget);
```

Features, limits & formats [16]

Interaction capabilities [16.1]

```
#define
  PM_INTERACTION_GRAPH_EDGES_NOT_APPLICABLE
  0u
#define
PM_INTERACTION_GRAPH_EDGES_SIMULATION_CAPACITY
PM_INVALID_INDEX
typedef struct PmInteractionCapabilityKey {
    PmInteractionFamily firstFamily;
    PmInteractionFamily secondFamily;
    PmInteractionScope scope;
    PmInteractionMode mode;
    PmInteractionDirection direction;
    PmInteractionMaterialProfile
      materialProfile;
} PmInteractionCapabilityKey;
typedef struct PmInteractionCapability {
    PmInteractionCapabilityKey key;
    uint32_t maxGraphEdgesPerSimulation;
} PmInteractionCapability;
PmResult
pmEnumeratePhysicalDeviceInteractionCapabilities(
    PmPhysicalDevice physicalDevice,
      uint32_t* pCount,
    PmInteractionCapability* pCapabilities);
PmResult
  pmGetPhysicalDeviceInteractionCapability(
    PmPhysicalDevice physicalDevice,
    const PmInteractionCapabilityKey* pKey,
    PmInteractionCapability* pCapability);
```

Physical-device limits [16.2]

```
typedef struct PmPhysicalDeviceLimits {
    PmDeviceSize maxAllocationSize;
    PmDeviceSize maxBufferSize;
    PmDeviceSize nonCoherentAtomSize;
    uint32_t maxMemoryAllocations;
    uint32_t maxRigidBodies;
    uint32_t maxRigidConstraints;
    uint32_t maxContacts;
    uint32_t maxFluidSystems;
    uint32_t maxParticlesPerFluid;
    uint32_t maxNeighborsPerParticle;
    uint32_t maxRigidIterations;
    uint32_t maxLiquidIterations;
    uint32_t maxSubsteps;
    uint32_t maxQueriesPerPool;
    uint64_t maxTimelineValueDifference;
} PmPhysicalDeviceLimits;
void pmGetPhysicalDeviceLimits(
    PmPhysicalDevice physicalDevice,
    PmPhysicalDeviceLimits* pLimits);
```

Format support query [16.3]

```
enum {
    PM_FORMAT_GEOMETRY_VERTEX_BIT = 0x1,
    PM_FORMAT_GEOMETRY_INDEX_BIT = 0x2,
    PM_FORMAT_PARTICLE_INPUT_BIT = 0x4,
    PM_FORMAT_SIMULATION_OUTPUT_BIT = 0x8
};
typedef struct PmFormatProperties {
    uint32_t elementSize;
    uint32_t elementAlignment;
    PmFlags supportedUsages;
} PmFormatProperties;
void pmGetPhysicalDeviceFormatProperties(
    PmPhysicalDevice physicalDevice,
    PmFormat format,
    PmFormatProperties* pProperties);
```

Portable buffer layouts [16.4]

R32_UINT	4 bytes / alignment 4 / unsigned index
R32_SFLOAT	4 bytes / alignment 4 / IEEE binary32
R32G32B32_SFLOAT	12 bytes / alignment 4 / x,y,z
R32G32B32A32_SFLOAT	16 bytes / alignment 4 / x,y,z,w
PARTICLE_STATE	28 bytes / alignment 4 / PmParticleState
FLUID_PARTICLE_OUTPUT	48 bytes / alignment 4 / PmFluidParticle Output
RIGID_STATE	52 bytes / alignment 4 / PmRigidState

Dependency checklist [16.5]

Rigid contacts	rigidBodies
Rigid joints	rigidBodies + rigidConstraints
One liquid	particleLiquids
Liquid ↔ bodies	both domains + liquidRigidCoupling
Liquid ↔ liquid	particleLiquids + liquidLiquidCoupling
GPU timing	timestampQueries + queue timestamp bits
Solver counters	statisticsQueries
Native object sharing	Per-backend import capabilities
Typed Vulkan adapters	vulkanInterop + required native features

Fundamental declarations [18]

Types, handles & constants [18.1]

```
#define PM_API_VERSION_0_2
  PM_MAKE_API_VERSION(0, 2, 0)
#define PM_WHOLE_SIZE UINT64_MAX
#define PM_TIMEOUT_INFINITE UINT64_MAX
#define PM_INVALID_INDEX UINT32_MAX
#define PM_QUEUE_FAMILY_IGNORED UINT32_MAX
#define PM_QUEUE_FAMILY_EXTERNAL (UINT32_MAX
  - 1u)
#define PM_FALSE 0u
#define PM_TRUE 1u
typedef uint32_t PmBool32;
typedef uint32_t PmFlags;
typedef uint64_t PmDeviceSize;
typedef uint64_t PmStageFlags;
typedef uint64_t PmAccessFlags;
typedef struct PmInstance_T* PmInstance;
typedef struct PmPhysicalDevice_T*
  PmPhysicalDevice;
typedef struct PmDevice_T* PmDevice;
typedef struct PmQueue_T* PmQueue;
typedef struct PmDeviceMemory_T*
  PmDeviceMemory;
typedef struct PmBuffer_T* PmBuffer;
typedef struct PmGeometry_T* PmGeometry;
typedef struct PmSimulation_T* PmSimulation;
typedef struct PmPhysicsPipeline_T*
  PmPhysicsPipeline;
typedef struct PmCommandPool_T*
  PmCommandPool;
typedef struct PmCommandBuffer_T*
  PmCommandBuffer;
typedef struct PmSemaphore_T* PmSemaphore;
typedef struct PmFence_T* PmFence;
typedef struct PmQueryPool_T* PmQueryPool;
```

Structure types [18.2]

```
typedef enum PmStructureType {
    PM_STRUCTURE_TYPE_UNDEFINED = 0,
    PM_STRUCTURE_TYPE_INSTANCE_CREATE_INFO =
      1,
    PM_STRUCTURE_TYPE_DEVICE_CREATE_INFO =
      2,
    PM_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO =
      3,
    PM_STRUCTURE_TYPE_BUFFER_CREATE_INFO =
      4,
    PM_STRUCTURE_TYPE_GEOMETRY_CREATE_INFO =
      5,
    PM_STRUCTURE_TYPE_SIMULATION_CREATE_INFO
    = 6,
PM_STRUCTURE_TYPE_PHYSICS_PIPELINE_CREATE_INFO
```

Structure types (continued) [18.2]

```
= 7,
PM_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO =
  8,
PM_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO
 = 9,
    PM_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO
     = 10,
    PM_STRUCTURE_TYPE_SUBMIT_INFO = 11,
PM_STRUCTURE_TYPE_NATIVE_DEVICE_IMPORT_INFO
 = 12,
    PM_STRUCTURE_TYPE_QUERY_POOL_CREATE_INFO
    = 13,
PM_STRUCTURE_TYPE_VALIDATION_DIAGNOSTICS_CREATE_INFO
= 14,
    PM_STRUCTURE_TYPE_COUPLING_STATE_INFO =
      15,
PM_STRUCTURE_TYPE_FLUID_PAIR_MATERIAL_STATE
 = 16,
PM_STRUCTURE_TYPE_PHYSICS_PIPELINE_COUPLING_STATE_INFO
= 17,
    PM_STRUCTURE_TYPE_FLUID_INTERFACE_STATE
     = 18,
PM_STRUCTURE_TYPE_SIMULATION_PROFILE_CREATE_INFO
= 19,
    PM_STRUCTURE_TYPE_SOFT_BODY_STATE_INFO =
      20,
PM_STRUCTURE_TYPE_SOFT_BODY_SELF_CONTACT_STATE
= 21,
} PmStructureType;
```

Symbol index

PFN_pmVoidFunction 1
PmAccessFlags 8
pmAllocateCommandBuffers 6
PmAllocateFn 1
pmAllocateMemory 2
PmAllocationCallbacks 1
PmAttachmentCouplingState 3
PmAxisLimit 3
PmAxisMotor 3
PmAxisSpring 3
pmBeginCommandBuffer 6
pmBindBufferMemory 2
pmBindGeometryMemory 2
pmBindSimulationMemory 2
PmBool32 8
PmBuffer 8
PmBufferCopy 6
PmBufferCreateInfo 2
PmBufferMemoryBarrier 6
PmBufferRange 2
PmBufferView 2
PmClothCapacity 2
PmClothInfo 4
PmClothMaterialState 4
PmClothVertexState 4
pmCmdApplyRigidInputs 3
pmCmdBeginStatisticsQuery 7
pmCmdBindPhysicsPipeline 6
pmCmdBuildGeometry 2
pmCmdClearPaintFields 5
pmCmdCopyBuffer 6
pmCmdCopyClothState 4
pmCmdCopyClothSurface 4
pmCmdCopyConstraintStates 5
pmCmdCopyContactEvents 5
pmCmdCopyDebugData 5
pmCmdCopyFluidParticles 6
pmCmdCopyPaintField 5
pmCmdCopyRigidStates 6
pmCmdCopyRopeState 4
pmCmdCopySmokeState 5
pmCmdCopySoftBodyState 5
pmCmdCopySoftBodySurface 5
pmCmdEndStatisticsQuery 7
pmCmdPipelineBarrier 6
pmCmdQueryHitBox 3
pmCmdResetQueryPool 7
pmCmdResetSimulation 6
pmCmdSetCouplings 3
pmCmdSetCouplings2 3
pmCmdSetFluidParticles 6
pmCmdSetKinematicTargets 3
pmCmdSetRigidStates 3
pmCmdStepSimulation 6
pmCmdWriteCloths 4
pmCmdWritePaintFields 5
pmCmdWritePaintRules 5
pmCmdWriteParticleDestroyers 5
pmCmdWriteParticleEmitters 5
pmCmdWriteRigidBodies 3
pmCmdWriteRigidConstraints 3
pmCmdWriteRopes 4
pmCmdWriteSmoke 5
pmCmdWriteSoftBodies 5
pmCmdWriteSoftBodies2 5
pmCmdWriteTimestamp 7
PmCommandBuffer 8
PmCommandBufferBeginInfo 6
PmCommandPool 8
PmCommandPoolCreateInfo 5
PmConstraintFrame 3
PmConstraintState 5
PmConstraintType 3
PmContactEvent 5
PmContactEventType 5
PmCouplingParameters 3
PmCouplingState 3
PmCouplingStateInfo 3
PmCouplingType 3
pmCreateBuffer 2
pmCreateCommandPool 5
pmCreateDevice 2
pmCreateFence 6
pmCreateGeometry 2
pmCreateInstance 1
pmCreatePhysicsPipeline 4
pmCreateQueryPool 7
pmCreateSemaphore 6
pmCreateSimulation 2
PmCrossingDirection 5
PmDebugRecord 5
PmDebugRecordType 5
pmDestroyBuffer 2
pmDestroyCommandPool 5
pmDestroyDevice 2
pmDestroyFence 6
pmDestroyGeometry 2
pmDestroyInstance 1
pmDestroyPhysicsPipeline 4

pmDestroyQueryPool 7
pmDestroySemaphore 6
pmDestroySimulation 2
PmDevice 8
PmDeviceCreateInfo 2
PmDeviceMemory 8
PmDeviceQueueCreateInfo 2
PmDeviceSize 8
pmDeviceWaitIdle 6
PmDomainOutputHeader 4
pmEndCommandBuffer 6
pmEnumerateDeviceExtensionProperties 1
pmEnumerateExecutionProfiles 1
pmEnumerateInstanceExtensionProperties 1
pmEnumerateInstanceLayerProperties 1
pmEnumerateInstanceVersion 1
pmEnumeratePhysicalDeviceInteractionCapabilities 7
pmEnumeratePhysicalDevices 1
PmExecutionProfileProperties 1
PmExtensionProperties 1
PmFence 8
PmFlags 8
PmFloat2 4
PmFloat3 4
PmFluidCapacity 2
PmFluidInterfaceState 3
PmFluidPairMaterialState 3
PmFluidParticleOutput 6
pmFlushMappedMemoryRanges 2
PmFormat 2
PmFormatProperties 7
pmFreeCommandBuffers 6
PmFreeFn 1
pmFreeMemory 2
PmGeometry 8
PmGeometryCreateInfo 2
pmGetBufferMemoryRequirements 2
pmGetDeviceMemoryBudget 7
pmGetDeviceProcAddr 1
pmGetDeviceQueue 2
pmGetFenceStatus 6
pmGetGeometryMemoryRequirements 2
pmGetInstanceProcAddr 1
pmGetNativeBuffer 7
pmGetNativeDevice 6
pmGetNativeQueue 6
pmGetNativeSemaphore 7
pmGetPhysicalDeviceFeatures 1
pmGetPhysicalDeviceFormatProperties 7
pmGetPhysicalDeviceInteractionCapability 7
pmGetPhysicalDeviceInteropProperties 6
pmGetPhysicalDeviceLimits 7
pmGetPhysicalDeviceMemoryProperties 2
pmGetPhysicalDeviceProperties 1
pmGetPhysicalDeviceQueueFamilies 1
pmGetQueryPoolResults 7
pmGetSemaphoreCounterValue 6
pmGetSimulationMemoryRequirements 2
pmGetSimulationStatus 7
pmGetVulkanBufferEXT 7
pmGetVulkanDeviceEXT 7
pmGetVulkanSemaphoreEXT 7
PmHitBox 3
PmHitBoxOutputHeader 3
PmHitBoxParticle 3
pmImportNativeBuffer 7
pmImportNativeDevice 6
pmImportNativeSemaphore 7
pmImportVulkanBufferEXT 7
pmImportVulkanDeviceEXT 7
pmImportVulkanSemaphoreEXT 7
PmInstance 8
PmInstanceCreateInfo 1
PmInteractionCapability 7
PmInteractionCapabilityKey 7
PmInteractionDirection 3
PmInteractionFamily 3
PmInteractionMaterialProfile 3
PmInteractionMode 3
PmInteractionModeFlags 3
PmInteractionScope 3
pmInvalidateMappedMemoryRanges 2
PmLayerProperties 1
PmLiquidMaterialState 3
pmMapMemory 2
PmMappedMemoryRange 2
PmMechanicalCouplingState 3
PmMemoryAllocateInfo 2
PmMemoryBarrier 6
PmMemoryBudget 7
PmMemoryBudgetConfidence 7
PmMemoryHeap 2
PmMemoryProperties 2
PmMemoryRequirements 2
PmMemoryType 2
PmMotionType 3
PmNativeBackend 6
PmNativeBufferInfo 7
PmNativeDeviceImportInfo 6
PmNativeDeviceInfo 6
PmNativeHandle 6
PmNativeInteropProperties 6
PmNativeQueueInfo 6
PmPaintFieldCapacity 2
PmPaintFieldInfo 5
PmPaintRuleInfo 5
PmPaintSourceType 5
PmPaintTargetType 5
PmParticleDestroyerInfo 5
PmParticleEmitterInfo 5
PmParticleState 6
PmPhysicalDevice 8
PmPhysicalDeviceFeatures 1
PmPhysicalDeviceLimits 7
PmPhysicalDeviceProperties 1
PmPhysicsPipeline 8
PmPhysicsPipelineCouplingStateInfo 4
PmPhysicsPipelineCreateInfo 4
PmQueryPool 8
PmQueryPoolCreateInfo 7
PmQueryType 7
PmQueue 8
PmQueueFamilyProperties 1
pmQueueSubmit 6
pmQueueWaitIdle 6
pmResetCommandBuffer 6
pmResetCommandPool 5
pmResetFences 6
PmResult 1
PmRigidBodyInfo 3
PmRigidConstraintInfo 3
PmRigidInput 3
PmRigidSolverState 3
PmRigidState 3
PmRopeAttachment 4
PmRopeCapacity 2
PmRopeInfo 4
PmRopeMaterialState 4
PmRopeNodeState 4
PmSemaphore 8
PmSemaphoreCreateInfo 6
PmSemaphoreSubmitInfo 6
PmSimulation 8
PmSimulationCreateInfo 2
PmSimulationMemoryBinding 2
PmSimulationMemoryRequirements 2
PmSimulationProfileCreateInfo 1
PmSmokeCapacity 2
PmSmokeCouplingState 3
PmSmokeInfo 5
PmSmokeMaterialState 4
PmSmokeParticleState 5
PmSoftBodyBond 4
PmSoftBodyCapacity 2
PmSoftBodyInfo 5
PmSoftBodyMaterialState 4
PmSoftBodyNodeState 5
PmSoftBodySelfContactState 5
PmSoftBodyStateInfo 5
PmSoftBodySurfaceBinding 4
PmStageFlags 8
PmStepInfo 6
PmStepStatistics 7
PmStructureType 8
PmSubmitInfo 6
PmThermalCouplingState 3
pmUnmapMemory 2
PmValidationDiagnostic 1
PmValidationDiagnosticFn 1
PmValidationDiagnosticsCreateInfo 1
PmValidationObjectType 1
PmValidationSeverity 1
PmVulkanBufferImportInfo 7
PmVulkanDeviceImportInfo 7
PmVulkanDeviceInfo 7
PmVulkanQueueInfo 7
pmWaitForFences 6
pmWaitSemaphore 6